

CS 650 – Full Stack Application Develop

5-2 Activity: End-to-End Request Tracing

Malgorzata Debska

Southern New Hampshire University

Brian Enochson

August 5, 2026

Debugging a Full Stack Application

Reproducing the Functional Error

The first step in the debugging process was reproducing the issue that users were experiencing. I launched the application in my Codio environment and opened the wedding RSVP page in the browser. To recreate the problem, I completed every field in the RSVP form by entering a guest name, email address, attendance status, meal preference, and a message for the couple. After completing the form, I clicked the Send RSVP button.

Instead of successfully submitting the RSVP, the application displayed an HTTP 404 error message. The guest's information was not added to the guestbook, and no confirmation message appeared. From the user's perspective, the submission appeared to fail completely, making it impossible to RSVP through the application. This confirmed that the reported issue could be reproduced consistently.

RSVP form displaying the HTTP 404 error after clicking Send RSVP

The screenshot shows a web browser window with the title "Wedding RSVP & Guestbook" and the address bar displaying "localhost:5173". The main content area features a white "RSVP" form. At the top of the form, a red error message "HTTP 404" is displayed. Below this, the form contains several fields: "Name" with the value "Malgorzata", "Email" with the value "margaretdebska30@gmail.com", and a "Will you attend?" section with radio buttons for "Joyfully accepts" (selected) and "Regretfully declines". There is also a checkbox for "Bringing a plus one" and a "Meal Preference" dropdown menu set to "Chicken". A text area for "Leave a message for the couple" contains the text "Thank you!". At the bottom of the form is a red "Send RSVP" button. Below the form, the word "Guestbook" is visible.

Hypothesizing the Source of the Error

After reproducing the error, I formed an initial hypothesis about where the failure was occurring. Because the application allowed me to enter information into the form without any problems, I concluded that the user interface itself was functioning correctly. The issue seemed to occur only after the form attempted to communicate with the server.

Based on these observations, I believed that the problem involved both the front-end and back-end layers. My initial assumption was that the front end was sending the request to an incorrect

API endpoint or that the server was missing the expected route. Since the browser displayed an HTTP 404 error, which indicates that a requested resource cannot be found, I suspected that the application was attempting to reach an endpoint that did not exist.

To verify this hypothesis, I reviewed both the front-end code responsible for submitting the RSVP form and the server-side API routes. This comparison helped identify whether the endpoint used by the browser matched the routes implemented on the back end.

Tracing the Expected Request Flow

In a properly functioning application, the request should follow a predictable path through several layers of the system. First, the user enters their RSVP information into the form and clicks the Send RSVP button. The front-end application then collects the data entered by the user and creates an HTTP POST request containing the RSVP information as a JSON payload. That request is sent to the backend API. The back-end server receives the request through the appropriate API endpoint, validates the submitted information, stores the RSVP in the database, and returns a successful response to the browser. Finally, the front end displays a confirmation message to the user and updates the guestbook with the newly submitted RSVP.

Using the browser Developer Tools, I examined the actual request generated by the application. The Network tab showed the following information:

- **Endpoint:** /api/rsvp
- **HTTP Method:** POST
- **Status Code:** 404 Not Found

because the endpoint itself was incorrect. The expected behavior was for the application to send the request to the correct endpoint, where the server would successfully process the RSVP, save it to the database, and return a success response. Instead, the request failed immediately because the requested endpoint could not be found. This prevented the remainder of the request flow from occurring.

Implementing the Correction

After tracing the request through the application, I identified the root cause of the problem. The front end attempted to submit RSVP information to the endpoint `/api/rsvp`, while the back end expected requests to be sent to `/api/rsvps`. Because these routes did not match, every submission resulted in a 404 Not Found response before the server could process any data. To resolve the issue, I updated the front-end code so that it submitted requests to the correct endpoint, `/api/rsvps`, matching the API route implemented on the server. While reviewing the application, I also identified additional issues that could affect future functionality. The back-end success response was updated to return the correct HTTP success status after saving an RSVP, and the meal preference field was fully supported throughout the application by updating both the database schema and the server logic responsible for storing RSVP records. After implementing these changes, I restarted the application and tested the RSVP process again. This time, the submission was completed successfully. The application displayed a confirmation message thanking the guest for submitting the RSVP, and the guestbook immediately displayed the newly submitted RSVP information, including the selected meal preference.

Successful RSVP submission showing the confirmation message and updated guestbook

Wedding RSVP & Guestbook

localhost:5173

Bella & Edward

Sunday, August 13, 2006 · The Cullen Mansion

RSVP

Thank you for your RSVP!

Name

Email

Will you attend?

☒ Joyfully accepts

☐ Regretfully declines

☐ Bringing a plus one

Meal Preference

Select...

Leave a message for the couple

Write your wishes...

Send RSVP

Guestbook

Test Guest

Meal: vegetarian

"Congratulations!"

Attending

Margaret D

Meal: fish

"Thank you!"

Attending

I chose this approach because it corrected the actual source of the problem rather than attempting to work around it. Matching the front-end request with the existing back-end API maintained a consistent architecture and required only a small modification to restore proper communication between both layers. An alternative approach would have been to modify the server by creating

an additional route for `/api/rsvp` that redirected or duplicated the existing functionality. Although this would also have resolved the immediate error, maintaining duplicate endpoints would increase maintenance effort and could introduce confusion for future developers. Updating the front end to use the correct endpoint was the cleaner and more maintainable solution.

Summary of the Debugging Process

My original hypothesis was mostly correct. I suspected that the problem involved communication between the front end and back end, and further investigation confirmed that the application was sending requests to an incorrect API endpoint. Using the browser Developer Tools made it possible to verify exactly where the request failed and why the server returned a 404 error. This debugging exercise improved my understanding of how all layers of a full stack application work together. Before completing this activity, I primarily viewed the front end and back end as separate components. Tracing the request from the browser to the API and then to the database demonstrated that even a small mismatch, such as an incorrect endpoint name, can prevent an entire feature from functioning. I also learned the importance of verifying the complete request lifecycle instead of focusing only on the visible error presented to the user. Browser Developer Tools, particularly the Network tab, provided valuable information about request methods, endpoints, status codes, and server responses, making it much easier to identify the true source of the problem. This activity reinforced that effective debugging requires examining the entire application architecture, understanding how information moves between components, and verifying that every layer communicates correctly with the next.

